Image Processing Mini-Project

A Comparative Study of RAG Techniques for Generating Image Captions

Phase 1 — Data Exploration and Pre-processing

Technical report

Prepared by:

Meradi Serine Feriel

Seba Bouchra

Aggoune Abderrahim

Souchane Serine

Academic Year: 2026/2027

Contents

1 Introduction and objectives of the phase **3**

2 Project organisation and reproducibility **3**

2.1 Pipeline overview . . . 3

2.2 Repository layout . . . 4

2.3 Reproducibility measures . . . 4

3 Datasets and splitting strategy **4**

3.1 Datasets . . . 4

4 Step 1 — Data audit and validation **5**

4.1 Objective of the dataset audit . . . 5

4.2 Method (`audit_dataset.py`) . . . 5

4.3 Results . . . 6

4.4 Interpretation . . . 6

5 Statistical exploration of the dataset **7**

5.1 Objective and scope . . . 7

5.2 Method . . . 7

5.3 Distribution of object classes . . . 7

5.4 Scene complexity . . . 9

5.5 Textual statistics . . . 11

5.6 Diversity analysis . . . 13

5.7 Exploration of the knowledge base . . . 14

5.8 Summary of the exploration . . . 16

6 Pre-processing steps **16**

6.1 Step 2 — Split creation and anti-leakage control . . . 16

6.2 Step 3 — Image pre-processing . . . 17

6.3 Step 4 — Caption pre-processing . . . 19

6.4 Step 5 — Metadata extraction . . . 20

6.5 Step 6 — Embedding generation . . . 22

6.6 Step 7 — Knowledge-base construction . . . 23

6.7 Step 8 — Quality control . . . 24

6.8 Sanity validation with the retrieval pipeline . . . 25

7 Validation and freezing of the release **27**

7.1 Validation subset and selection of the pre-processing configuration . . . 27

7.2 Frozen artefacts 28
7.3 Freezing rules 28

8 Discussion **28**

9 Conclusion **28**

1 Introduction and objectives of the phase

Automatic image captioning is a core task at the crossroads of computer vision and natural language processing. Modern Vision–Language Models (VLMs) generate fluent captions, but they may *omit* objects that are present, *invent* objects that are absent (hallucination), or lack precision about attributes and relations. Retrieval-Augmented Generation (RAG) grounds the generation process in external evidence (similar images, reference captions, metadata), with the goal of improving accuracy, relevance and visual grounding [4].

The overall mini-project compares ten RAG techniques (naive/dense, sparse BM25, hybrid, reranking, multimodal, contextual compression, Self-RAG, corrective RAG, query expansion and metadata-filtered RAG) on a subset of MS COCO [1]. The purpose of this first phase is therefore to transform the raw COCO data into a *clean, reproducible and frozen* set of artefacts.

This report documents, step by step and in the order imposed by the project statement (Sections 4.3 and 4.4):

1. the **statistical exploration** of the dataset (object classes, scene complexity, textual statistics, diversity, visualisations);
2. the **eight pre-processing steps** (plus the optional YOLO detection branch added to Step 5 and the extension of the test set to 1 000 images): (1) data audit and validation, (2) split creation and anti-leakage control, (3) image pre-processing, (4) caption pre-processing, (5) metadata extraction, (6) embedding generation, (7) knowledge-base construction, (8) quality control;
3. the **validation and freezing** of the release.

For every step we describe the working method (script, algorithm, parameters), report the results obtained from the code outputs and generated files, and interpret them.

2 Project organisation and reproducibility

2.1 Pipeline overview

The Data Preparation and Knowledge Base Construction Pipeline is implemented using a set of independent Python scripts, each responsible for a specific task. These scripts process the dataset through a series of stages, generating intermediate outputs and artefacts required by subsequent steps. Figure 1 summarises the flow of data.

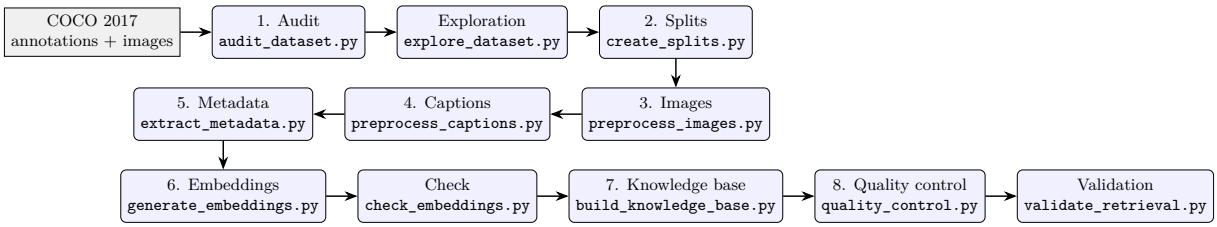


Figure 1: The Data Preparation and Knowledge Base Construction Pipeline (Each box is an independent script; each arrow is a file dependency).

2.2 Repository layout

Directory	Content
`scripts/`	25 Python scripts: the 11 pipeline scripts of Figure 1, plus `explore_knowledge_base.py` (knowledge-base exploration), `create_updated_test_split.py` and `check_current_splits.py` (test-set extension and split check), and the YOLO scripts (pilot, threshold experiment, full extraction, merge, checksums, tests).
`data/train2017/`, `data/val2017/`	Placeholders for the COCO 2017 images (not versioned; to be downloaded).
`data/annotations/`	Official COCO 2017 JSON files (captions, instances, person keypoints).
`data/splits/`	`knowledge_base_image_ids.json`, `validation_image_ids.json`, `test_image_ids.json`.
`data/processed_images/`	Pre-processed knowledge-base images (RGB, EXIF-corrected).
`data/captions/`	`kb_captions_raw.json` and `kb_captions_cleaned.json`.
`data/metadata/`	Objects, bounding boxes and spatial relations of the knowledge base (COCO ground truth).
`data/exploration/`	Outputs of the knowledge-base exploration (POS, colours, shapes, indoor/outdoor, day/night, COCO categories).
`data/visual_detections/`, `data/visual_detections_test/`	YOLO detections: 100-image pilot, full 10 000-image extraction, threshold and COCO-comparison experiments.
`data/embeddings/`	CLIP image and text embeddings (`.npy`) with ID mappings.
`data/index/`	FAISS indexes and the knowledge-base descriptor.
`reports/`	Audit, exploration, pre-processing, quality-control, retrieval-validation and YOLO-merge reports.

Table 1: Repository layout.

2.3 Reproducibility measures

- **Public repository**: https://github.com/Serine205/rag-coco-image-captioning.
- **Fixed random seed** (`SEED = 42`) for all random selections.
- **Frozen splits** stored as versioned JSON files of image IDs.
- **Relative paths** computed from the script location (`Path(__file__)`), so the project runs on any machine.
- **Every script writes a report** (JSON/CSV) so that each decision is traceable.
- **Git LFS** (`data/** filter=lfs`; every file under `data/` must be fetched with `git lfs pull`) is used for the large artefacts (annotations, embeddings, indexes); the raw and processed images are excluded from version control by `.gitignore` to avoid unnecessarily increasing the repository size, as the original COCO images can be downloaded again from the official dataset.

3 Datasets and splitting strategy

3.1 Datasets

Two official splits of COCO 2017 are used:

- **COCO train2017**: 118 287 images and 591 753 captions, from which the 10 000-image

knowledge base and the 100-image tuning set are sampled;
- **COCO val2017**: 5 000 images and 25 014 captions, from which the evaluation (test) images are sampled.

Usage	Source	Images	Output file
RAG knowledge base	train2017	10 000	`knowledge_base_image_ids.json`
Hyper-parameter tuning (validation)	train2017 (disjoint from KB)	100	`validation_image_ids.json`
Test / evaluation	val2017	1 000	`test_1000_image_ids.json`
Legacy test (kept)	val2017	100	`test_image_ids.json`, `test_100_image_ids_legacy.json`

Table 2: Splits implemented by `create_splits.py` (the 1 000-image test file is produced by `create_updated_test_split.py`).

Size of the test set

The project statement is not consistent on this point: Sections 4.1 and 4.2 specify **100** test images, whereas Sections 4.4.2 and 4.4.3 mention **1 000**. The repository was first built with 100 test images and then extended: `create_updated_test_split.py` keeps the 100 original images (saved unchanged as `test_100_image_ids_legacy.json`) and adds 900 images drawn at random (seed 42) from the remaining val2017 images, giving `test_1000_image_ids.json`. The script asserts that the 1 000 images belong to val2017, include the legacy 100, and that the knowledge-base and validation sets are disjoint. The file `test_image_ids.json` was deliberately left untouched (still 100 images). The 1 000-image file is therefore the reference test set of this release, while the figures of the formal quality-control report (Table 15) were produced with the 100-image file.

4 Step 1 — Data audit and validation

4.1 Objective of the dataset audit

Before starting data pre-processing and implementing the Retrieval-Augmented Generation (RAG) approaches, it is important to verify the quality and consistency of the MS COCO dataset, to ensure that the images and their corresponding annotations are correctly organised and that no structural problem could affect the subsequent stages of the project.

Several checks were performed to verify the completeness and consistency of the raw data: image counts, number of reference captions per image, missing, corrupted and duplicated images, and image–caption integrity (Section 4.4.3, step 1).

The audit results generated by `scripts/audit_dataset.py` were saved in a JSON report named `dataset_audit.json`, located in the `reports/` directory. This report contains the audit results for both datasets.

4.2 Method (`audit_dataset.py`)

For each of the two datasets, the script:

1. loads the COCO caption JSON and counts the physical `.jpg` files in the image directory;
2. checks the uniqueness of image IDs (`duplicate_ids`);

3. counts the captions per image (`Counter` over `image_id`) and builds the distribution of this count; images whose count differs from 5 are flagged;
4. for each image listed in the annotations, builds the file name `{id:012d}.jpg`; a missing file is recorded as *missing*; otherwise the file is opened with PIL and `Image.verify()` is called – an exception marks the image as *corrupted*;
5. computes the **SHA-256 hash of each file**; identical hashes reveal **exact duplicate images**;
6. detects *orphan captions*, i.e. captions whose `image_id` is absent from the image list.

4.3 Results

Indicator	**train2017**	**val2017**
Physical images	118 287	5 000
Images in annotations	118 287	5 000
Total captions	591 753	25 014
Duplicate image IDs	0	0
Images with exactly 5 captions	117 972	4 987
Images with 6 captions	312	12
Images with 7 captions	3	1
Images with $\neq 5$ captions	315	13
Missing images	0	0
Corrupted images	0	0
Groups of exact duplicate images (SHA-256)	12	0
Orphan captions	0	0

Table 3: Dataset audit (source: `reports/dataset_audit.json`).

4.4 Interpretation

Interpretation

- **Completeness.** Both datasets are complete: the number of physical images equals the number of images declared in the annotations, with no missing, corrupted or orphan item. No sample had to be removed, so the rule "remove invalid samples only if documented" is trivially satisfied.
- **Captions per image.** The "5 captions per image" rule holds for 99.73 % of train2017 (117 972/118 287) and 99.74 % of val2017 (4 987/5 000). The few exceptions have *more* captions (6 or 7), never fewer: $591\,753 - 5 \times 118\,287 = 318$ extra captions $= 312 \times 1 + 3 \times 2$, which matches the table exactly. No image lacks references.
- **Consequence for the project.** Because some images carry 6 or 7 captions, the statement's "50 000 captions in the knowledge base" becomes 50 028 in practice (Section 6.3). We deliberately keep *all* reference captions rather than truncating to 5: truncating would silently discard human annotations and would require an arbitrary choice.
- **Duplicates.** 12 groups of byte-identical images exist in train2017 (none in val2017). Their effect is negligible at the scale of 118 287 images, but a duplicate present both in the knowledge base and in the tuning set would be a subtle form of leakage.

5 Statistical exploration of the dataset

5.1 Objective and scope

The statistical exploration aims to provide a documented analysis covering: (i) object-class distribution (top 50), (ii) scene complexity, (iii) textual statistics, (iv) diversity analysis, (v) visualisations. The analysis is implemented in `explore_dataset.py` and is applied to the **complete COCO train2017 set** (118 287 images), which is the population from which the knowledge base is drawn. Working on the full population gives stable statistics and shows what a random 10 000-image sample is expected to look like. All generated results, including CSV files, the JSON summary, and visualisations, are saved in the `reports/exploration/` directory.

5.2 Method

- **Captions** (`captions_train2017.json`): the caption length is the number of tokens found by the regular expression `\b\w+\b`; for the vocabulary and the total word count, captions are lower-cased and words are extracted with `[a-zA-Z]+` (a different tokenisation from the one used for the caption length, hence slightly different totals). We then compute the total word count, the vocabulary size, the mean / median / min / max length and the word frequencies (`collections.Counter`).
- **Objects** (`instances_train2017.json`): each annotation carries a `category_id` that is mapped to its name; we count annotations per category, annotations per image, and the set of categories present in each image.
- **Co-occurrence**: for each image, every unordered pair of distinct categories is counted once (`itertools.combinations`); the 20 most frequent pairs are exported.
- **Outputs**: CSV tables (`top_50_objects.csv`, `top_words.csv`, `object_cooccurrence.csv`), a JSON summary (`exploration_summary.json`) and four 300-dpi figures.

5.3 Distribution of object classes

COCO annotates **80 object categories**; train2017 contains **860 001 object annotations**. Table 4 gives the top 50 categories and Figure 2 shows the top 20.

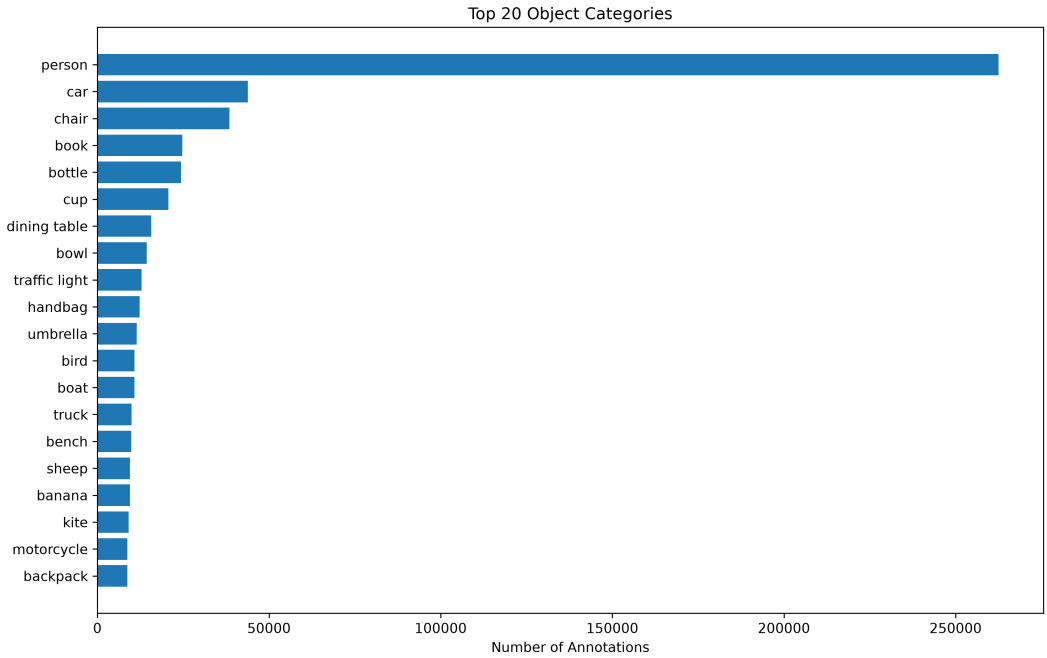


Figure 2: Top 20 object categories in COCO train2017 (number of annotations).

Table 4: Top 50 object categories (number of annotations, train2017).

#	**Category**	**Freq.**	#	**Category**	**Freq.**	#	**Category**	**Freq.**
1	person	262 465	18	kite	9 076	35	cake	6 353
2	car	43 867	19	motorcycle	8 725	36	sports ball	6 347
3	chair	38 491	20	backpack	8 720	37	clock	6 334
4	book	24 715	21	potted plant	8 652	38	suitcase	6 192
5	bottle	24 342	22	cow	8 147	39	spoon	6 165
6	cup	20 650	23	wine glass	7 913	40	surfboard	6 126
7	dining table	15 714	24	carrot	7 852	41	bus	6 069
8	bowl	14 358	25	knife	7 770	42	apple	5 851
9	traffic light	12 884	26	broccoli	7 308	43	pizza	5 821
10	handbag	12 354	27	donut	7 179	44	tv	5 805
11	umbrella	11 431	28	bicycle	7 113	45	couch	5 779
12	bird	10 806	29	skis	6 646	46	remote	5 703
13	boat	10 759	30	vase	6 613	47	sink	5 610
14	truck	9 973	31	horse	6 587	48	skateboard	5 543
15	bench	9 838	32	tie	6 496	49	elephant	5 513
16	sheep	9 509	33	cell phone	6 434	50	dog	5 508
17	banana	9 458	34	orange	6 399			

Interpretation

- **Very strong class imbalance (long tail).** The class *person* alone accounts for 262 465/860 001 = **30.5** % of all annotations, i.e. 2.2 persons per image on average and about 6 times more than the second class (car, 43 867). The top 3, top 5 and top 10 classes cover 40.1 %, 45.8 % and 54.6 % of annotations; the top 50 cover 88.8 %, so the 30 remaining classes share only 11.2 %.
- **Everyday-life dominance.** The most frequent classes are urban and domestic objects (car, chair, book, bottle, cup, dining table, traffic light, handbag). Typical COCO scenes are therefore streets, kitchens, dining rooms and sports fields.
- **Implication for captioning and RAG.** A captioner and a retriever will see "person" evidence in a majority of images; the retrieved neighbours will tend to share this class, which can make retrieval look good for trivial reasons. For hallucination analysis (Section 10.4 of the statement), *rare* classes (the tail of the distribution) are where a model is most likely to omit or invent objects, so error rates should be reported per frequency band, not only globally.
- **Metadata filtering (G28–G30).** For metadata-filtered RAG, a filter on "person" is almost non-selective, whereas a filter on a tail class (e.g. elephant, 5 513 annotations) is very selective; the choice of filtering strategy (strict vs. boosting) will thus depend strongly on the class frequency.

5.4 Scene complexity

Objects per image. The mean number of object annotations per image is 860 001/118 287 = **7.27**. Figure 3 shows the distribution.

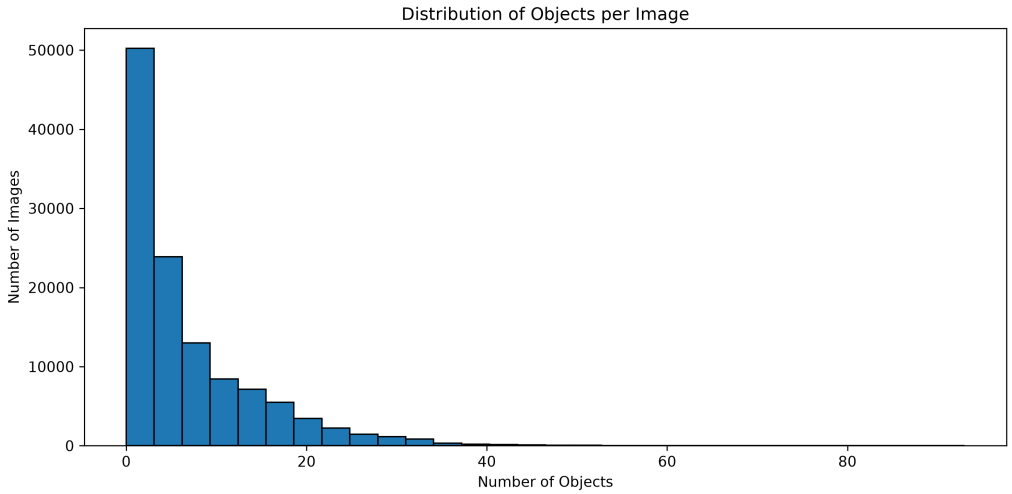


Figure 3: Distribution of the number of annotated objects per image (train2017).

Interpretation

The distribution is strongly **right-skewed** (long tail up to about 90 objects): the first bin (0–3 objects) contains roughly 50 000 images, i.e. about 42 % of the dataset, and the frequency decreases quickly afterwards (about 24 000 images in the 3–6 bin, 13 000 in the 6–9 bin). The mean (7.27) is therefore larger than the typical (modal) scene, which contains only a few objects. COCO thus mixes *simple scenes* (a few salient objects) and *complex scenes* (crowds, kitchens, streets with tens of instances). This confirms that the test subset must be **stratified** on scene complexity, as required by the statement, otherwise a random 100-image sample could be dominated by simple scenes and hide failures on cluttered ones. Complex scenes are also where *object hallucination* and *omission* are most likely.

Object co-occurrence and spatial relations. Table 5 reports the 20 most frequent unordered pairs of categories appearing together in the same image.

#	Object 1	Object 2	Images	#	Object 1	Object 2	Images
1	car	person	8 519	11	person	sports ball	4 097
2	chair	person	8 035	12	person	truck	3 992
3	handbag	person	6 222	13	cell phone	person	3 830
4	dining table	person	5 869	14	car	truck	3 664
5	chair	dining table	5 124	15	person	tie	3 632
6	backpack	person	5 034	16	person	umbrella	3 447
7	cup	person	4 848	17	person	skateboard	3 417
8	cup	dining table	4 679	18	bowl	dining table	3 396
9	bottle	person	4 561	19	person	surfboard	3 365
10	bench	person	4 108	20	person	tennis racket	3 361

Table 5: Top 20 object co-occurrences (number of images containing both categories).

Interpretation

Co-occurrences reflect **real-world contexts**: person+car / person+handbag (street), chair+dining table+cup+bowl (indoor dining), person+sports ball / skateboard / surfboard / tennis racket (sports), car+truck (road traffic). 16 of the 20 pairs include "person", which is expected given its frequency. These strong contextual priors are a double-edged sword for captioning: they let a model infer plausible objects (useful for RAG), but also cause *hallucination by association* (e.g. mentioning a "chair" because a "dining table" is visible). They also justify the **spatial relations** extracted in Step 5 (Section 6.4): left/right and above/below relations are computed between the bounding boxes of co-occurring objects.

5.5 Textual statistics

Statistic	Value
Total captions	591 753
Total words	6 199 120
Vocabulary size (distinct lower-cased words)	26 466
Mean caption length (words)	10.48
Median caption length (words)	10.0
Minimum / maximum caption length	5 / 50
Type/token ratio (26 466/6 199 120)	0.43 %
Captions per image (mean)	5.003

Table 6: Caption statistics on train2017 (source: `exploration_summary.json`).

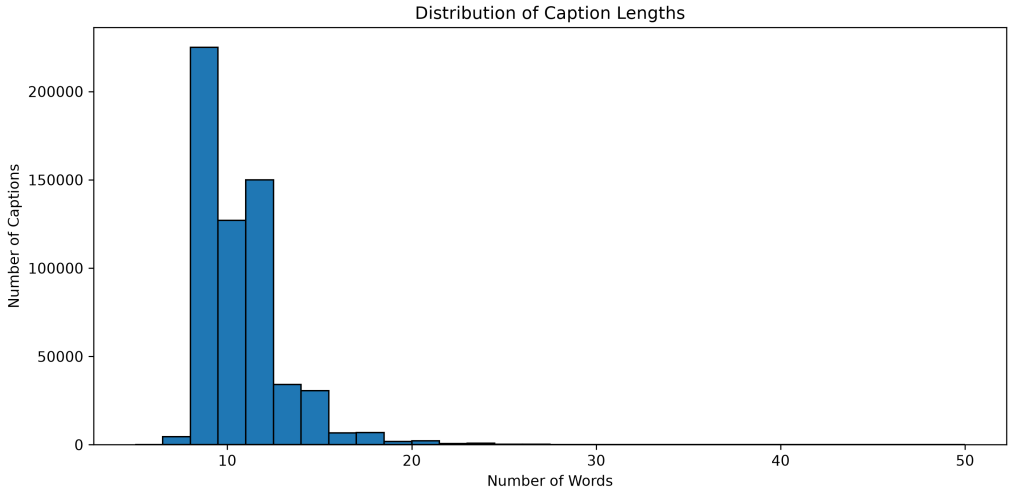


Figure 4: Distribution of caption lengths (words).

Interpretation

- **Short, homogeneous captions.** Mean 10.48 and median 10 words, with the bulk between 8 and 13 words (histogram modes at 9–10 and 12). The minimum is 5 and the long tail ends at 50 words, but captions above 20 words are very rare. This is consistent with COCO's annotation protocol (one sentence describing the main scene).
- **Compatibility with the encoder.** CLIP's text encoder accepts at most 77 tokens. Since almost all captions are under 20 words, truncation (`max_length=77`, `truncation=True`) can only affect a handful of extreme outliers (the longest caption has 50 words, which may approach the 77-token limit once punctuation and sub-word splitting are counted).
- **Reference length for generation.** A generated caption is expected to be about 10 words long. Models that produce much longer captions will be penalised by BLEU/-CIDEr (length mismatch), so the generation length should be controlled during hyper-parameter tuning.
- **Low lexical diversity ratio.** The vocabulary (26 466 words) is small relative to the 6.2 M tokens (type/token ratio 0.43 %): captions reuse a restricted, repetitive vocabulary. This favours lexical retrieval such as BM25 on the knowledge-base captions, because query words are likely to occur in the corpus.

Most frequent words. Figure 5 and Table 7 give the most frequent words.

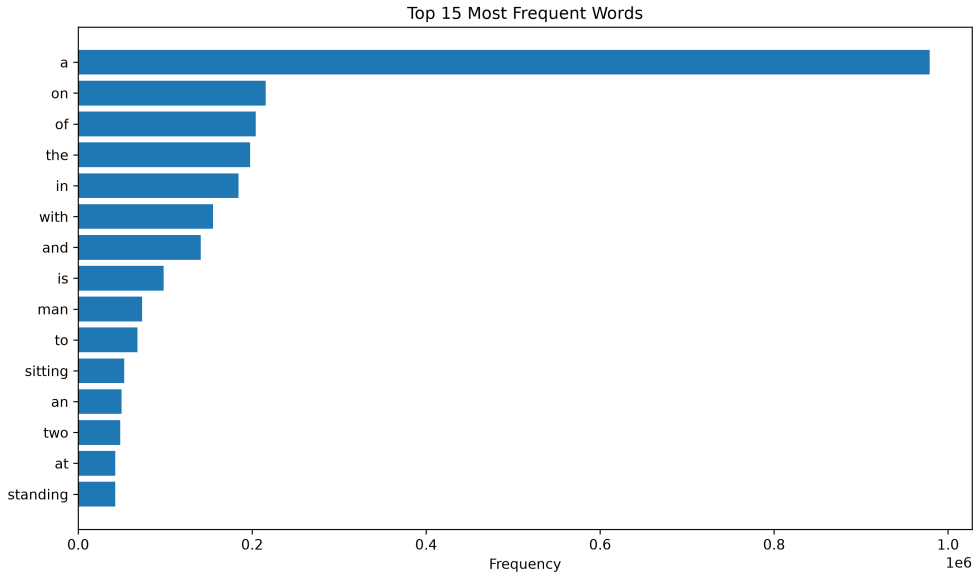


Figure 5: Top 15 most frequent words in captions (frequency $\times 10^6$).

#	Word	Freq.	#	Word	Freq.	#	Word	Freq.
1	a	978 685	11	sitting	52 911	21	street	30 291
2	on	215 689	12	an	49 875	22	table	30 187
3	of	204 072	13	two	48 404	23	that	28 175
4	the	197 771	14	at	42 724	24	holding	27 968
5	in	184 143	15	standing	42 485	25	it	26 981
6	with	154 811	16	people	41 939	26	person	24 745
7	and	140 916	17	are	41 086	27	large	24 556
8	is	98 184	18	next	37 215	28	some	24 330
9	man	73 340	19	white	36 364	29	down	23 140
10	to	68 161	20	woman	34 210	30	top	22 299

Table 7: Top 30 words in the reference captions (train2017).

Interpretation

- **Function words dominate.** The article "a" alone represents 15.8 % of all tokens; the top 10 words represent 37.4 % and the top 30 represent 48.5 %. In the list, 12 of the first 20 words are function words (articles, prepositions, conjunctions, auxiliaries). This has direct consequences for **BM25 and sparse retrieval**: stop-words carry almost no discriminative information and must be handled by IDF weighting or removed in the sparse index.
- **Content words reveal the dataset's semantics.** Among content words, *man*, *woman*, *people*, *person* (persons), *sitting*, *standing*, *holding* (actions/postures), *two* (counting), *white* (colour), *street*, *table*, *top* (places and spatial terms) dominate. Captions therefore describe mostly **people, their posture, their location and a few attributes**, which agrees with the object statistics ("person" = 30.5 % of annotations).
- **Spatial and relational vocabulary.** The words *on*, *in*, *at*, *next* (to), *down*, *top* (of) show that captions encode spatial relations frequently: relational hallucinations (wrong preposition / position) are a real risk to evaluate.

5.6 Diversity analysis

The statement asks for an analysis of attributes (colour, shapes) and contexts (indoor/outdoor, day/night). No pixel-level attribute classifier is used: colours and shapes are extracted from the captions and the indoor/outdoor and day/night contexts are predicted with CLIP (Section 5.7). Combined with the object statistics, the diversity is characterised as follows:

- **Object diversity**: 80 categories, with a strong long tail (see above), plus 7.27 objects per image on average.
- **Context diversity**: besides the CLIP predictions of Section 5.7, indoor contexts are indicated by categories and words such as *chair*, *dining table*, *couch*, *sink*, *tv*, *book*, *cup*, *bowl*; outdoor contexts by *car*, *traffic light*, *bench*, *boat*, *kite*, *surfboard*, *skis*, *elephant*, *street*. Both families are well represented in the top 50, which indicates a balanced mixture of indoor and outdoor scenes rather than one dominant setting.
- **Attribute diversity**: the only colour word in the top-30 words is *white* (36 364 occurrences, rank 19), and the size adjective *large* (rank 27). The other colours appear beyond the 30 exported words; COCO captions therefore contain attributes, but in a minor proportion relative to objects and actions.
- **Visual diversity of the knowledge base**: Section 6.2 reports the distribution of image resolutions and aspect ratios of the 10 000 KB images.

5.7 Exploration of the knowledge base

The statistics above describe the whole of train2017. A second script, `explore_knowledge_base.py`, analyses the 10 000 knowledge-base images and their 50 028 cleaned captions and covers what the first script does not: part-of-speech (POS) distribution, colours, shapes, and indoor/outdoor and day/night contexts. Outputs are stored in `data/exploration/` (CSV and JSON) and the figures in `reports/exploration/`.

- **POS distribution**: spaCy (`en_core_web_sm`) tags every caption token (spaces and punctuation excluded); nouns, verbs and adjectives are counted by lemma (top 20 each).
- **Colours and shapes**: lexical extraction from the captions with fixed vocabularies (11 colours, 8 shapes, with synonyms such as *grey*/*gray*). These are the colours *mentioned* by annotators, not measured on pixels.
- **Indoor/outdoor and day/night**: zero-shot classification of each image with CLIP ViT-B/32 (cosine similarity between the stored image embedding and two text prompts; the higher score gives the label). These labels are predictions, not verified ground truth.
- **COCO categories**: categories, super-categories, objects per image and category co-occurrences computed on the knowledge base (files `coco_*.csv`).

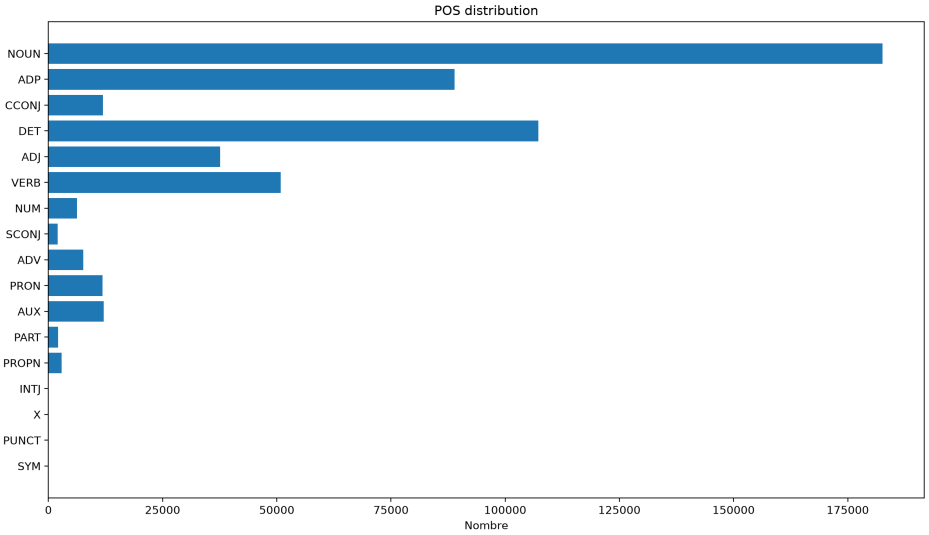


Figure 6: POS distribution of the knowledge-base captions (spaCy).

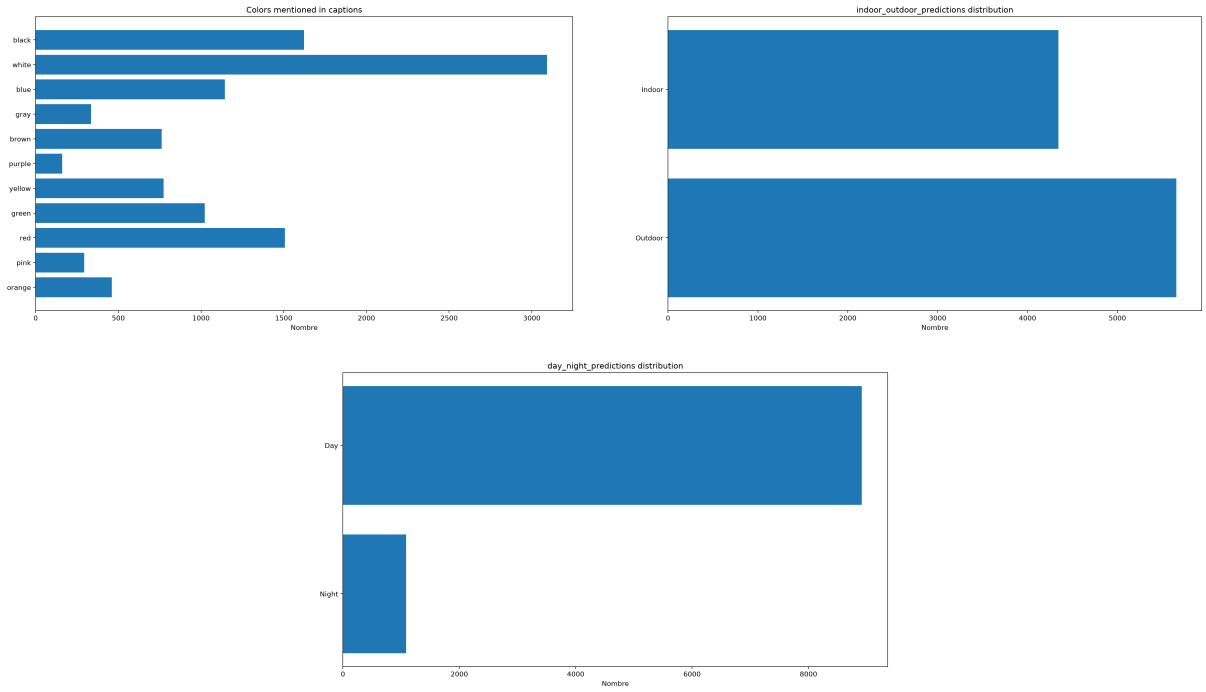


Figure 7: Colours mentioned in captions (top left), CLIP indoor/outdoor prediction (top right) and CLIP day/night prediction (bottom).

Interpretation

- **Noun-centred captions.** Nouns are by far the most frequent tag (about 180 000 tokens), followed by determiners (about 107 000), adpositions (about 89 000), verbs (about 51 000) and adjectives (about 38 000). This confirms the word-level findings of the previous section: captions name objects and place them with prepositions, so object and relation hallucinations are the natural errors to measure.
- **Colours.** *White* is the most frequent colour word (about 3 100 mentions), well ahead of *black*, *red*, *blue* and *green* (about 1 000 to 1 600 each); *purple*, *pink* and *gray* are rare. Attribute hallucinations will be easier to measure for the frequent colours than for the rare ones.
- **Contexts.** CLIP predicts roughly 57 % outdoor and 43 % indoor images, and about 89 % day against 11 % night. Indoor and outdoor scenes are both well represented, but **night scenes are rare**: results on night images will rest on few examples and should not be over-interpreted.
- **Precision of these values.** The counts quoted in this subsection are read from the figures, hence the "about"; the exact values are in the JSON files of `data/exploration/` (Git LFS).

5.8 Summary of the exploration

Property	Finding
Object classes	80 classes, 860 001 annotations; "person" = 30.5 %; top 50 = 88.8 %.
Scene complexity	7.27 objects/image on average; strongly right-skewed, many simple scenes and a long tail of cluttered ones.
Captions	10.48 words on average (median 10, range 5–50); 26 466 distinct words.
Vocabulary	Dominated by function words (top 30 = 48.5 % of tokens); content centred on people, postures and locations.
Contexts	Balanced indoor and outdoor scenes (about 43 % / 57 % predicted by CLIP on the knowledge base); night scenes are rare (about 11 %).
Attributes	Colours mentioned in captions are dominated by *white*; nouns dominate the POS distribution.

Table 8: Key exploration findings.

6 Pre-processing steps

6.1 Step 2 — Split creation and anti-leakage control

Objective. Create and freeze three disjoint splits (knowledge base, validation, test) and guarantee that no test reference caption can ever be retrieved or used (anti-leakage rule of Section 4.2).

Method (`create_splits.py`).

1. Load the image IDs of train2017 and val2017 and check that they are unique (an exception is raised otherwise).
2. Check sizes: train2017 must contain at least 10 000 + 100 images, val2017 at least N_{test}.
3. Set `random.seed(42)`, shuffle the train2017 IDs, take the first 10 000 as the **knowledge base** and the next 100 (positions 10 000–10 099) as the **validation** set. Because the two slices of the same shuffled list cannot overlap, the validation images are disjoint from the knowledge base by construction.
4. Shuffle the val2017 IDs and take the first 100 as the **test** set. Test images come from a *different official split* than the knowledge base, so they cannot appear in it.
5. Execute three `assert` statements verifying that the three ID sets are pairwise disjoint.
6. Save the three image-ID lists as JSON files in `data/splits/`: knowledge_base_image_ids.json, validation_image_ids.json, and test_image_ids.json.
7. **Test-set extension** (`create_updated_test_split.py`): add 900 random val2017 images to the original 100 to obtain `test_1000_image_ids.json`, and keep the original list as `test_100_image_ids_legacy.json` (see the remark of Section 3). `check_current_splits.py` re-verifies the provenance (knowledge base and validation in train2017, test in val2017) and the pairwise overlaps without modifying any file.

Check	Value	Status
Knowledge-base images	10 000	PASS
Validation images	100	PASS
Test images	100	PASS
Duplicate IDs inside each split	0 / 0 / 0	PASS
KB ∩ validation	0	PASS
KB ∩ test	0	PASS
Validation ∩ test	0	PASS

Table 9: Split verification (source: `quality_control_report.json`, run with the 100-image test file; to be re-run for `test_1000_image_ids.json`).

Interpretation

The two sampling strategies give two layers of protection: (i) *ID disjointness* (verified by assertion, and again by the quality-control script), and (ii) *source separation* (test images are drawn from val2017, the knowledge base from train2017). Moreover, only the captions of the 10 000 knowledge-base images are ever loaded into the text index (Step 4), so **no test caption can be retrieved** – the quality-control step checks this explicitly (0 test or validation IDs in the text embeddings). Fixing the seed makes the splits perfectly reproducible and identical for all 30 groups once the ID files are shared. For the extended test set, source separation (val2017 versus train2017) gives the same guarantee for the 900 added images, but the formal `quality_control.py` audit has not yet been re-run on them.

6.2 Step 3 — Image pre-processing

Objective. Obtain, for the 10 000 knowledge-base images, RGB images with a correct orientation, ready for the image encoder, while keeping the original aspect ratio.

Method (`preprocess_images.py`). For each ID of the knowledge base:

1. open `train2017/{id:012d}.jpg` and record the original width and height;
2. apply `ImageOps.exif_transpose` to correct the orientation stored in the EXIF metadata;
3. convert to RGB mode (`convert("RGB")`) so that grey-scale or palette images have three channels;
4. record the processed size and save the image as JPEG (quality 95) in `data/processed_images/`;
5. catch any exception per image and log it in an `errors` list, so that one failure does not stop the process.

No resizing or cropping is performed in this script: resizing and normalisation depend on the assigned image encoder, so they are delegated to the encoder's own processor (`CLIPProcessor`) in Step 6. For CLIP ViT-B/32 this processor resizes the shortest side to 224 pixels (bicubic), centre-crops to 224×224 and normalises with the CLIP channel means and standard deviations.

Indicator	Value
Images to process	10 000
Successful / failed	10 000 / 0
Colour mode after processing	RGB for 10 000 images
Images whose size changed after EXIF correction	0
Width: min / mean / max (px)	150 / 579.3 / 640
Height: min / mean / max (px)	120 / 483.3 / 640
Mean aspect ratio (width/height)	1.26
Landscape / portrait / square images	7 352 / 2 306 / 342
Most frequent size	640 × 480 (2 208 images, 22.1 %)
Other frequent sizes	640 × 427 (1 246), 480 × 640 (733), 640 × 426 (473), 500 × 375 (447)

Table 10: Image pre-processing results (source: `image_preprocessing_report.json`, computed over the 10 000 images).

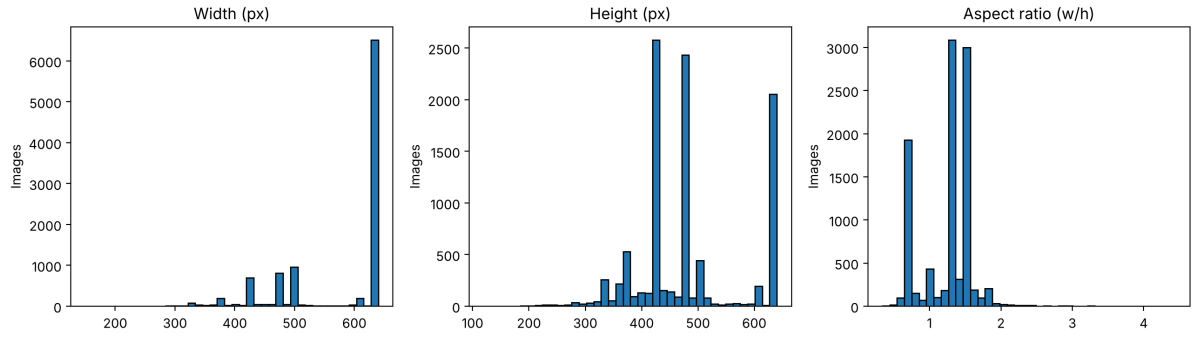


Figure 8: Distribution of width, height and aspect ratio of the 10 000 knowledge-base images.

Interpretation

- **100 % success, no information loss in geometry.** No image failed, all are RGB, and none changed size after the EXIF transposition: none of these 10 000 images carried a rotation tag that changes its dimensions, so this operation acts as a safeguard rather than a correction. The pipeline is nevertheless robust to datasets that do.
- **Heterogeneous formats.** COCO images have at most 640 pixels on the long side, with typical sizes 640 × 480 or 640 × 427. About 73.5 % of the images are landscape, 23.1 % portrait and 3.4 % square. Because CLIP centre-crops to a square, **the left/right borders of landscape images are cut off** (about 25 % of the width for a 4:3 image). Objects located near the borders may therefore be invisible to the encoder; this is a known limitation of CLIP-based retrieval and a possible cause of omissions.
- **Low resolution of the source.** The images are small (579×483 on average), so the processed JPEGs are already below the native resolution that modern VLMs use; this limits the visibility of small objects, which are the most frequently hallucinated or omitted ones.
- **Re-encoding.** Saving at JPEG quality 95 introduces a tiny additional compression artefact relative to the original files. It is imperceptible for the encoder but should be mentioned for strict reproducibility.

6.3 Step 4 — Caption pre-processing

Objective. Provide both the *raw* reference captions (unchanged, used for evaluation) and a *cleaned* version used only for retrieval and indexing, stored with distinct field and file names to avoid evaluation leakage.

Method (`preprocess_captions.py`).

1. Load `captions_train2017.json` and the knowledge-base IDs.
2. Keep only the annotations whose `image_id` belongs to the knowledge base (a set lookup, $O(1)$ per annotation).
3. For each kept annotation, store the **raw** caption unchanged (`kb_captions_raw.json`) and a **cleaned** caption (`kb_captions_cleaned.json`), both keeping `annotation_id` and `image_id`.
4. The cleaning function applies: (a) lower-casing, (b) stripping leading/trailing spaces, (c) collapsing multiple whitespace characters (`\s+` → single space), (d) removing spaces before punctuation (`\s+([.,!?;:])`).
5. Validate: every knowledge-base image has captions, the number of captions per image is reported, and the raw and cleaned lists have the same length.

Punctuation is *not* removed, and no explicit tokenisation is stored: tokenisation is left to the encoder (CLIP's byte-pair-encoding tokenizer, which handles punctuation by itself) and to the sparse retrievers assigned to some groups.

Indicator	**Value**
Knowledge-base images	10 000
Raw captions / cleaned captions	50 028 / 50 028
Images without captions	0
Images with exactly 5 captions	9 972
Images with 6 captions	28
Images with fewer than 5 captions	0
Raw and cleaned counts match	`True`

Table 11: Caption pre-processing (source: `caption_preprocessing_report.json`).

Interpretation

- **50 028 instead of 50 000 captions.** 28 knowledge-base images have 6 captions (28 extra captions), in agreement with the audit (0.28 % of the images, a proportion close to the 0.27 % observed in the full train2017 set, i.e. 315/118 287). We keep all of them: the extra captions are valid human references, and removing them would require an arbitrary choice.
- **Evaluation integrity.** Because cleaning (lower-casing etc.) can change n-gram matches in BLEU/CIDEr, the evaluation must always use the raw captions, while retrieval uses the cleaned ones. Keeping two files with identical IDs makes this separation explicit and prevents accidental mixing.
- **Minimal cleaning is intentional.** Over-aggressive cleaning (stop-word removal, stemming) would damage dense embeddings, which are trained on natural sentences. The cleaning only removes formatting noise (case, spaces), so the semantics are preserved.
- **Alignment.** The same `annotation_id` appears in both versions, so each cleaned caption can be traced back to its raw original.

6.4 Step 5 — Metadata extraction

Objective. Attach to each knowledge-base image its visual metadata: detected objects, scene information and spatial relations, so that the metadata-filtered and multimodal RAG groups can use them.

Method (`extract_metadata.py`). Instead of running an object detector, the script uses the **ground-truth COCO instance annotations** (`instances_train2017.json`), which are permitted as "annotation fields" by the statement (Section 4.4.3, step 5). For every knowledge-base image it stores:

- image information: file name, width, height;
- the raw captions of the image;
- the list of objects: `annotation_id`, `category_id`, category name, `bbox` ($[x, y, w, h]$), `area`, `iscrowd`;
- `object_count`;
- **spatial relations** between every pair of objects.

Spatial relation rule. For two objects A and B with boxes (x, y, w, h), the centres are $c_A = (x_A + w_A/2,\ y_A + h_A/2)$ and c_B. The relations are

$$\text{left_of if } c_{A,x} < c_{B,x}, \qquad \text{right_of if } c_{A,x} > c_{B,x},$$
$$\text{above if } c_{A,y} < c_{B,y}, \qquad \text{below if } c_{A,y} > c_{B,y},$$

so each pair yields up to two relations (one horizontal and one vertical). Results are saved as `kb_metadata.json`, a per-image summary `kb_metadata_summary.csv` (image ID, file name, size, object count, caption count, number of relations) and a global statistics file `metadata_statistics.json` (total objects, mean objects per image, relation-type counts, category counts).

Optional detector branch (YOLO). The statement lists an object detector as optional. In addition to the ground-truth annotations, a YOLO detector (`yolo26n.pt`, confidence threshold 0.25, CPU) was run on the knowledge-base images, in four stages: a 100-image pilot (`pilot_yolo_100.py`, evaluated by `evaluate_yolo_pilot_100.py`); a threshold experiment over $\{0.25, 0.40, 0.50\}$ with an IoU threshold of 0.50 (`experiment_yolo_thresholds.py`, `compare_`

`yolo_coco.py`); the full extraction on the 10 000 images (`extract_yolo_10000.py`, checkpoint every 100 images); and the merge with the COCO metadata (`merge_yolo_metadata.py`). The merged file `kb_metadata_with_yolo.json` adds three fields to each image (`yolo_detections`, `yolo_detection_count`, `yolo_detected_categories`) and preserves the COCO fields. SHA-256 checksums of inputs and outputs are stored in `reports/yolo_merge_sha256.json`. The merged file and the weights are excluded from Git (`.gitignore`) because of their size and are regenerated by the scripts.

Indicator (source: `yolo_metadata_merge_report.json`)	**Value**
Model / threshold / device	`yolo26n.pt` / 0.25 / CPU
Images merged (unique image IDs)	10 000
Total detections	49 375
Mean detections per image	4.94
Detected categories (out of 80)	79
Images without any detection	136 (1.4 %)
Merge quality-control flags (7)	all `true`

Table 12: Merge of the YOLO detections with the COCO metadata.

Two bounding-box conventions

COCO boxes are stored as $[x, y, w, h]$, YOLO boxes as $[x_1, y_1, x_2, y_2]$: any group that uses both sources must convert one convention to the other. YOLO detections are *predicted* objects and contain detection errors, unlike the COCO annotations: they are the realistic query-time source for the metadata-filtered groups (G28–G30), whereas the COCO annotations remain the oracle.

Interpretation

- **Ground truth as the primary source.** COCO annotations are human-verified, so this metadata has no detection noise, no confidence threshold to tune and no model version to document. This provides an *upper bound* (an "oracle") for metadata-based techniques; it also gives the reference needed to measure object hallucination later (is a generated object in the annotated list?).
- **Expected content.** Since the 10 000 images are a uniform random sample of train2017, their metadata should reflect the exploration of Section 5: about 7.3 objects per image and about 30 % of them persons.
- **Number of relations grows quadratically.** For an image with n objects, $\binom{n}{2}$ pairs and up to $2\binom{n}{2}$ relations are stored. A cluttered image with 90 objects yields several thousand relations, which is why the full metadata file is large and why this representation should be filtered (e.g. to the biggest or non-crowd objects) before being injected into a prompt.
- **Limits of the relation rule.** Comparing box centres is a coarse approximation: it does not capture "on", "next to", "in front of" or containment, and an object can be simultaneously "left_of" and "above" another one. These are *geometric* relations in the image plane, not semantic relations as expressed in captions.
- **Attributes.** COCO provides no attribute annotations (colour, material), so the "attributes" field requested by the statement is not populated by this script; attribute information can only come from the captions or from a separate model.

6.5 Step 6 — Embedding generation

Objective. Encode every knowledge-base image and every cleaned caption into a common vector space, so that image-to-image, image-to-text, text-to-image and text-to-text retrieval are all possible.

Model choice. CLIP ViT-B/32 [2] (`openai/clip-vit-base-patch32`, Hugging Face Transformers) is used as image encoder *and* text encoder. CLIP is trained contrastively so that matching image/text pairs have a high cosine similarity; this makes cross-modal retrieval possible with a single model and a single index dimension (512). It is one of the image encoders allowed by the statement (CLIP, SigLIP, DINOv2) and has the lowest computational cost of the CLIP family, an advantage on CPU-only hardware.

Method (`generate_embeddings.py`).

1. Load the knowledge-base IDs and the **cleaned** captions, keeping only captions of knowledge-base images; stop if an image has no caption or a processed image file is missing.
2. **Image embeddings**: batches of 32 images are converted by `CLIPProcessor`, passed through the vision transformer (`vision_model`, pooled output) and the visual projection layer; the 512-d vector is **L2-normalised**.
3. **Text embeddings**: batches of 32 captions are tokenised (padding, truncation to 77 tokens), passed through the text transformer and the text projection; the 512-d vector is **L2-normalised**.
4. Everything is computed under `torch.no_grad()` in evaluation mode; the device is chosen automatically (CUDA if available, otherwise CPU).
5. Save `image_embeddings.npy`, `text_embeddings.npy` and three ID arrays (`image_ids`, `text_image_ids`, `text_annotation_ids`) that link every row to its image/caption, together with `embedding_metadata.json` (model, dimension, counts, batch size, normalisation flags).

Property	**Value**
Model	`openai/clip-vit-base-patch32`
Embedding dimension	512
Image embeddings	$10\,000 \times 512$, `float32`
Text embeddings	$50\,028 \times 512$, `float32`
Batch size	32
Max text length	77 tokens
Normalisation	L2 (unit norm) for images and texts
Memory (images)	$10\,000 \times 512 \times 4$ B ≈ 19.5 MiB
Memory (texts)	$50\,028 \times 512 \times 4$ B ≈ 97.7 MiB

Table 13: Embedding configuration and sizes (sources: `embedding_metadata.json`, `quality_control_report.json`).

Interpretation

- **Why L2 normalisation.** After normalisation, the inner product equals the cosine similarity. This makes the similarity scores bounded in $[-1, 1]$, independent of vector norms, and allows the use of a fast inner-product index.
- **A shared space.** Because images and texts live in the same 512-d space, the *multimodal RAG* groups can retrieve captions with an image query, while the *dense RAG* groups can embed a textual query and search images or captions without re-encoding the knowledge base. All groups share this single encoder, so differences between techniques are not caused by different embeddings.
- **Image/text asymmetry.** There are five times more text vectors than image vectors (50 028 vs 10 000), since each image has about five captions. Text vectors therefore carry the dominant share of the memory and of the search cost.
- **Variants for ablations.** Ablation 5 of the statement (CLIP vs SigLIP vs DINOv2) and the embedding-model hyper-parameter will require regenerating these files with another encoder; the script's `MODEL_NAME` constant isolates this change.

6.6 Step 7 — Knowledge-base construction

Objective. Build the vector databases on which all retrieval techniques will operate, and document the required elements: index type, similarity metric, dimension, size and a link between each vector and its image ID, captions and metadata.

Method (`build_knowledge_base.py`).

1. Load the five embedding files and **validate** them: 2-D arrays, dimension 512, equal lengths between vectors and IDs, unique image IDs, absence of NaN/Inf, and every caption's image ID belongs to the knowledge base.
2. Re-normalise the vectors (`faiss.normalize_L2`) as a safety measure and cast them to contiguous `float32` arrays.
3. Build **two FAISS indexes**: an image index and a text (caption) index, both of type `IndexFlatIP`.
4. Save the indexes and the ID mapping files (`image_index_ids.npy`, `text_index_image_ids.npy`, `text_index_annotation_ids.npy`) and the descriptor `knowledge_base_metadata.json`.
5. Reload the indexes from disk and check that the number of stored vectors is unchanged.

Documented item	Value
Library	FAISS [3] 1.15.1
Index type	`IndexFlatIP` (flat, exhaustive search, no compression)
Similarity metric	Inner product = cosine similarity (normalised vectors)
Embedding dimension	512
Image index size	10 000 vectors
Text index size	50 028 vectors
Row → record link	Row i of the image index → `image_index_ids[i]`; row j of the text index → `text_index_image_ids[j]` and `text_index_annotation_ids[j]`; metadata through the image ID.

Table 14: Knowledge-base documentation.

Interpretation

- **Why a flat index.** With 10 000 images and 50 028 captions, an exhaustive search costs at most $50\,028 \times 512 \approx 2.6 \times 10^7$ multiply-adds per query, which is negligible (milliseconds). Approximate indexes (IVF, HNSW) would add training/hyper-parameter complexity and *approximation error* without any benefit at this scale. The flat index returns the exact nearest neighbours, so retrieval quality is not confounded by index approximation, which is essential for a fair comparison of RAG techniques and for measuring retrieval latency.
- **Inner product = cosine.** The `IndexFlatIP` on unit vectors ranks identically to a cosine-similarity search.
- **Separate image and text indexes.** They allow the four retrieval directions (I→I, I→T, T→I, T→T) needed by naive/dense, multimodal and hybrid RAG; the ID mapping converts any caption hit to its parent image, and vice versa.
- **Documentation gap.** The statement also requires the *indexing time*. The script prints the execution time on the console but does not save it to a report; for a flat index this is a few seconds, since no training is needed and the vectors are only copied into the structure.

6.7 Step 8 — Quality control

Objective. Detect missing embeddings, NaN values, duplicate IDs, caption mismatches and any data leakage before freezing the data.

Method (`quality_control.py`). The script performs ten independent checks, each recorded as `PASS`, `WARNING` or `FAIL` in `quality_control_report.json`:

1. **Splits**: sizes, duplicates inside each split, pairwise overlaps.
2. **Embedding files**: all five files exist.
3. **Embedding dimensions and types**: shapes $(10\,000, 512)$ and $(50\,028, 512)$, `float32`.
4. **NaN / infinite values**: none in image or text embeddings.
5. **Duplicate IDs**: image IDs and annotation IDs are unique.
6. **L2 normalisation**: minimum and maximum vector norms are within a tolerance of 1.
7. **Caption–image matching**: every caption points to a known image, every image has captions and embeddings, minimum/maximum captions per image.

8. **Processed images**: the 10 000 expected files exist.
9. **FAISS indexes**: vector counts and dimensions are as expected.
10. **Data leakage**: no overlap between splits and no validation/test image ID among the text-embedding image IDs.

Check	Status	Key values
Data splits	PASS	10 000 / 100 / 100 images; 0 duplicate; 0 overlap
Embedding files	PASS	5 files found
Embedding dimensions	PASS	image (10 000, 512); text (50 028, 512); `float32`
NaN and infinite values	PASS	0 invalid values (images and texts)
Duplicate IDs	PASS	0 duplicate image IDs; 0 duplicate annotation IDs
L2 normalisation	PASS	norms in [0.9999998, 1.0000002] (images and texts)
Caption–image matching	PASS	50 028 captions; 10 000 images with captions; captions per image: min 5, max 6; no missing/unknown
Processed images	PASS	10 000 expected, 0 missing
FAISS indexes	PASS	10 000 and 50 028 vectors; $d = 512$
Data leakage	PASS	0 / 0 / 0 overlaps; 0 validation/test IDs in text embeddings
Overall	**PASS**	10 checks: 10 passed, 0 failed, 0 warnings, 0 errors

Table 15: Quality-control report.

Interpretation

All ten checks pass with no warning. The normalisation deviation ($\leq 2.4 \times 10^{-7}$) is pure `float32` rounding. The caption statistics (9 972 images with 5 captions and 28 with 6) are consistent with the audit, the caption pre-processing report and the number of text vectors, which shows that the three representations (JSON, embeddings, index) have **consistent sizes**; the checks on IDs and the ID-mapping files then ensure that they are linked row by row. The leakage check is the most important: it confirms that the validation and test images are neither in the knowledge base nor in the text index, so evaluation on them is not biased by retrieval of their own captions.

6.8 Sanity validation with the retrieval pipeline

Objective. Section 4.4.4 requires that the frozen release *works with the retrieval pipeline*. A dedicated script (`validate_retrieval.py`) queries the knowledge base with the 100 **validation** images and their 500 captions, which are not in the knowledge base.

Method. The validation images are loaded from train2017 (EXIF-corrected, RGB), encoded with the same CLIP model and normalisation, and searched with $k = 10$ (CPU, batch size 16) in four configurations:

Experiment	Queries	Searched index
Image → Image	100 images	image index
Image → Text	100 images	text (caption) index
Text → Image	500 captions	image index
Text → Text	500 captions	text index

For each query, the ten best IDs and their cosine similarities are saved in `image_retrieval_results.json` and `caption_retrieval_results.json`. As the script itself states, no relevance judgements are available, so these results are *rankings and similarity scores*, not precision/recall.

Results. Mean, minimum and maximum of the **top-1 cosine similarity** per experiment:

Experiment	**Queries**	**Mean top-1**	**Min**	**Max**
Image → Image	100	0.814	0.579	0.946
Image → Text	100	0.328	0.283	0.391
Text → Image	500	0.325	0.237	0.393
Text → Text	500	0.889	0.565	1.000

Table 16: Top-1 similarity statistics of the validation retrieval (computed from the result files).

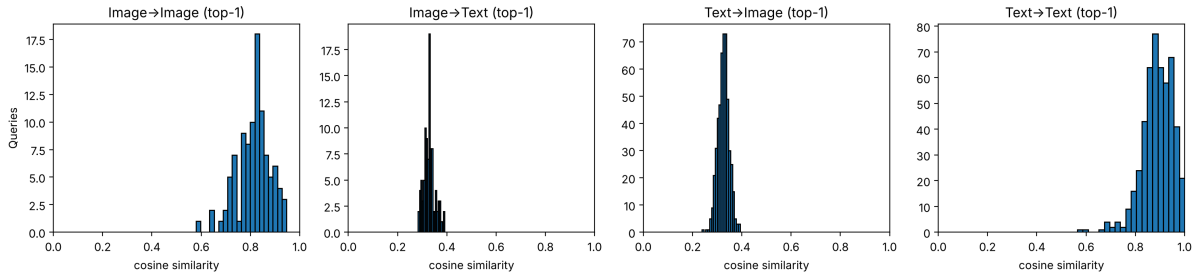


Figure 9: Distribution of the top-1 cosine similarity for the four retrieval directions.

Additional observations: the 10th image neighbour has a mean similarity of 0.759 (vs. 0.814 for the first), 85 of the 500 text queries have a top-1 similarity ≥ 0.95, and 9 reach exactly 1.0. For image-to-text, the ten retrieved captions come from 8.39 distinct images on average. Example: for the validation image 528231 (captions: "Two black and white cows standing on a lush green field.", ...), the best image neighbour has a similarity of 0.946.

Interpretation

- **The pipeline works end to end.** Each query returns 10 results with valid IDs, and the scores are consistent with CLIP's behaviour, which confirms that embeddings, indexes and ID mappings are correctly linked.
- **Modality gap.** Same-modality similarities (I→I ≈ 0.81, T→T ≈ 0.89) are about 2.5 times larger than cross-modal ones (I↔T ≈ 0.33). This is the well-known *modality gap* of CLIP: image and text embeddings occupy different regions of the sphere, and a cross-modal cosine of 0.3 is already a strong match. Consequently, **similarity thresholds must be chosen per modality** (the statement's thresholds 0.5, 0.7, 0.9 are meaningful for I→I or T→T, but would reject every cross-modal result), and scores from different directions must not be fused naively in hybrid RAG without normalisation.
- **Text-to-text exact matches.** The nine queries with similarity 1.0 correspond to validation captions that are *textually identical* (after lower-casing) to a caption in the knowledge base, e.g. "A train traveling down tracks next to a forest." or "A baseball player holding a bat during a game." This is not leakage (validation IDs are absent from the index), but a consequence of generic, repeated captions in COCO. It shows that text retrieval can return an identical sentence describing a *different* image – a reason to rely on image evidence as well and to treat caption similarity with caution.
- **Noise in the annotations.** One of the exact matches is the sentence "There is no image here to provide a caption for.", an annotator remark that is present in COCO. It confirms that a small amount of noisy captions exists in the data and could be retrieved as evidence.
- **Diversity of the retrieved captions.** The 8.39 distinct source images among 10 retrieved captions (image-to-text) show that retrieved captions mostly come from different images and are not dominated by the 5 captions of a single image, which is desirable for providing varied evidence.

7 Validation and freezing of the release

7.1 Validation subset and selection of the pre-processing configuration

The statement specifies that the 100 validation images are used to select the best pre-processing configuration before freezing. In this release, one pre-processing configuration is implemented (EXIF + RGB without resizing, minimal caption cleaning, CLIP ViT-B/32 with L2 normalisation, exact flat inner-product index). The validation subset was used to *verify* that the pipeline works (Section 6.8); no alternative configuration was compared on it.

7.2 Frozen artefacts

Frozen artefact	Content
`data/splits/*.json`	10 000 KB IDs, 100 validation IDs, 1 000 test IDs (`test_1000_image_ids.json`, seed 42) and the legacy 100-image test list.
`data/captions/kb_captions_raw.json`	50 028 raw captions (evaluation / generation context).
`data/captions/kb_captions_cleaned.json`	50 028 cleaned captions (retrieval and indexing only).
`data/processed_images/`	10 000 RGB, EXIF-corrected images.
`data/metadata/`	Objects, bounding boxes, spatial relations per image; YOLO detections in `data/visual_detections/` (merged file regenerated by `merge_yolo_metadata.py`).
`data/embeddings/`	$10\,000 \times 512$ image and $50\,028 \times 512$ text CLIP embeddings with ID arrays.
`data/index/`	FAISS `IndexFlatIP` image and text indexes with ID mappings.
`reports/`	Audit, exploration, pre-processing, QC and validation reports.

Table 17: Frozen release shared with all groups.

7.3 Freezing rules

- No group may modify the test split or the knowledge base.
- Test reference captions must never be used during retrieval or generation; they are only used by the metrics.
- A test image must never be retrieved for itself: this holds because test images are not indexed.
- The raw captions are used for evaluation; cleaned captions only for retrieval.

8 Discussion

The data-preparation phase yields a coherent and verified base: complete raw data (no missing or corrupted file), a reproducible disjoint split (10 000 / 100 / 100), a knowledge base of 10 000 images and 50 028 captions embedded with CLIP ViT-B/32 and indexed in exact FAISS indexes, and a quality-control report with 10/10 checks passed, including the leakage audit. The exploration shows that COCO is dominated by people in everyday scenes, with a strongly skewed number of objects per image, short and repetitive captions and a long-tailed object distribution; these properties will directly shape the retrieval and the hallucination analysis of the following phases.

9 Conclusion

This first phase delivered the complete, documented and frozen data foundation of the project. The audit confirmed the integrity of COCO 2017 (no missing or corrupted image; 99.7 % of images with exactly five captions); the exploration characterised the data (80 object classes with "person" at 30.5 % of annotations, 7.27 objects and 10.48 words per caption on average); the eight pre-processing steps, completed by a YOLO detection branch and an extended 1 000-image test set, produced a knowledge base of 10 000 images and 50 028 captions, embedded into a normalised

512-d CLIP space and indexed with exact FAISS inner-product indexes; and quality control (10/10 checks) together with a retrieval sanity test confirmed that the release is consistent, free of leakage and usable by all groups.

References

[1] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár and C. L. Zitnick, "Microsoft COCO: Common Objects in Context", in *ECCV*, 2014.

[2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision", in *ICML*, 2021.

[3] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs", *IEEE Transactions on Big Data* 7(3), 2021.

[4] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", in *NeurIPS*, vol. 33, 2020.